Functions

↳ take a piece of code
give it a name
reuse it anytime.

define a f^

```
def greet():
    print("Welcome user")
```

→ name → inputs will come

↳ indentation

```
def greet(name):
    print("Hello", name)
```

→ parameter

article → 100 books → 10% → $\frac{10}{100} \times 100 = 10$
90 books → ✓ → 100 − 10 = 90

$100 \times \frac{90}{100} = 90$

$100 \times \frac{90}{100} = 100 \times 0.9$

*args → any number of positional arguments.

```
def func(*args):
```

*args	**kwargs
values only	key=value
tuple	dictionary
(1,2,3)	{"name": "Sandfell"}

RECURSION → A f^ calls itself.
↳ 2 rules → Base condition (STOP)
↳ Recursive call.

lambda functions → one line anonymous f^

lambda parameters : expression

```
def add(a,b):
    return a+b
```

```
add = lambda a,b : a+b
```

map() → every element of a list.

[1, 2, 3, 4] → 2 → [2, 4, 6, 8]

loop.

map(function, iterable)
↓ what to apply
↳ list, tuple etc.

filter → filter elements based on conditions

even num.

x % 2 == 0

```
nums = [1, 2, 3, 4, 5]
result = filter(lambda x: x % 2 == 0, nums)
print(list(result))
```

[2, 4]

num(1, 2, 3, 4)

((1+2)+3)+4)

left, right = 0, len(list)

left, right = (0, len(list))
↓
tuple.

DECORATORS

↳ modifies another f^ without changing its code.

```
def decorator(func):
    def wrapper():
        print("before function")
        func()
        print("After function")
    return wrapper

def greet():
    print("Hello")

greet = decorator(greet)
greet()
```

pass greet into decorator.

func = greet

greet = wrapper.

Original greet → goes into the decorator

decorator → wraps it with extra behaviour

return wrapper

greet → now becomes wrapper.

Calling greet → actually calling wrapper()